Sales Prediction using Simple Linear Regression :

Objective:
Develop a simple linear regression model to predict sales based on advertising expenditure. The model analyzes the relationship between TV advertising spending and sales using the ISLR Advertising dataset.

Dataset:
Use the Advertising dataset from the ISLR package to analyse the relationship between TV advertising expenditure and sales. The objective is to build a simple linear regression model using TV advertising as the predictor variable for sales.

Exploring the Dataset

[10]:
```
# Supress Warnings

import warnings
warnings.filterwarnings('ignore')

# Import the numpy and pandas package

import numpy as np
import pandas as pd

# Data Visualisation
import matplotlib.pyplot as plt
import seaborn as sns
```

[8]:
```
advertising = pd.DataFrame(pd.read_csv("advertising.csv"))
advertising.head()
```

[7]:

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9

Dataset Inspection

[11]:
```
advertising.shape
```

[9]:
```
(200, 4)
```

[12]:
```
advertising.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   TV         200 non-null    float64
 1   Radio      200 non-null    float64
 2   Newspaper  200 non-null    float64
 3   Sales      200 non-null    float64
dtypes: float64(4)
memory usage: 6.3 KB
```

[13]:
```
advertising.describe()
```

[11]:

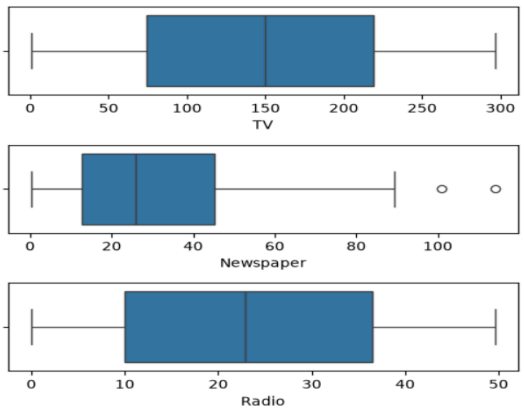
	TV	Radio	Newspaper	Sales
count	200.000000	200.000000	200.000000	200.000000
mean	147.042500	23.264000	30.554000	15.130500
std	85.854236	14.846809	21.778621	5.283892
min	0.700000	0.000000	0.300000	1.600000
25%	74.375000	9.975000	12.750000	11.000000
50%	149.750000	22.900000	25.750000	16.000000
75%	218.825000	36.525000	45.100000	19.050000
max	296.400000	49.600000	114.000000	27.000000

Data Cleaning

[14]:
```
# Checking Null values
advertising.isnull().sum()*100/advertising.shape[0]
# There are no NULL values in the dataset, hence it is clean.
```

[12]:
```
TV           0.0
Radio        0.0
Newspaper    0.0
Sales        0.0
dtype: float64
```

[21]:
```
# Outlier Analysis
fig, axs = plt.subplots(3, figsize = (5,5))
plt1 = sns.boxplot(x=advertising['TV'], ax = axs[0])
plt2 = sns.boxplot(x=advertising['Newspaper'], ax = axs[1])
plt3 = sns.boxplot(x=advertising['Radio'], ax = axs[2])
plt.tight_layout()
```



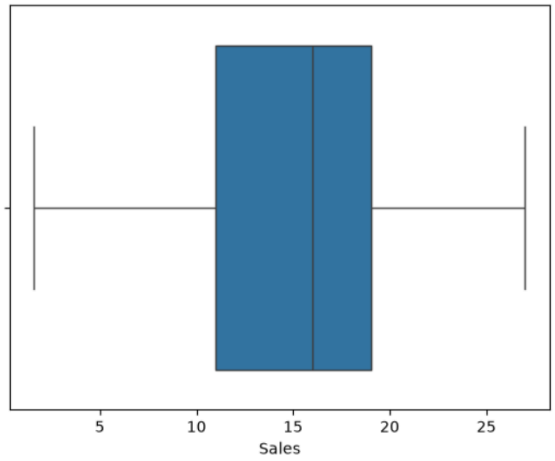
[48]:
```
#No significant outliers are observed in the dataset.
```

Exploratory Data Analysis

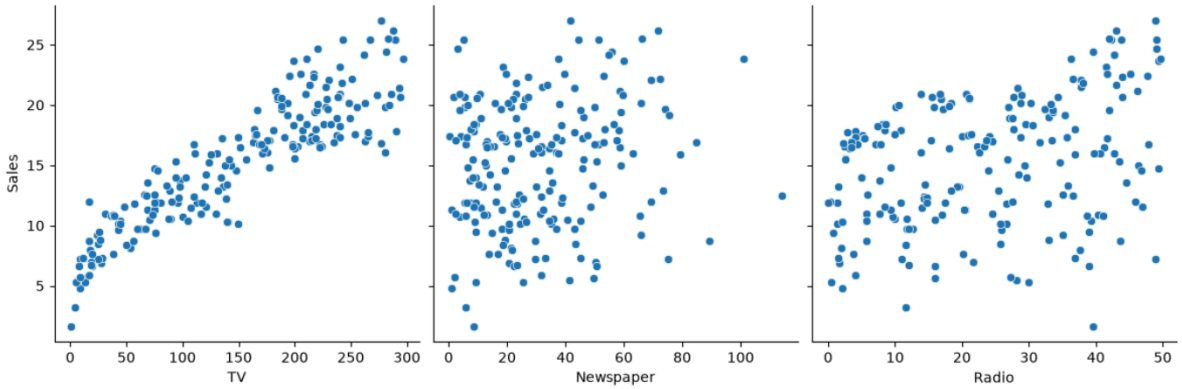
Univariate Analysis

Distribution of Sales

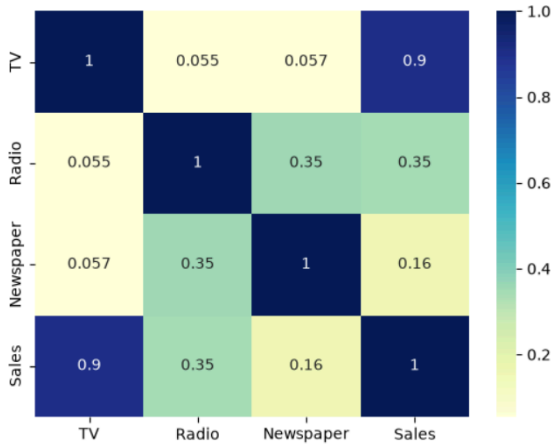
[23]:
```
sns.boxplot(x=advertising['Sales'])
plt.show()
```



[24]:
```
# Let's see how Sales are related with other variables using scatter plot.
sns.pairplot(advertising, x_vars=['TV', 'Newspaper', 'Radio'], y_vars='Sales', height=4, aspect=1, kind='scatter')
plt.show()
```



```
[25]: # Let's see the correlation between different variables.
sns.heatmap(advertising.corr(), cmap="YlGnBu", annot = True)
plt.show()
```



The pairplot and correlation heatmap indicate that TV advertising has the strongest relationship with Sales. Therefore, TV is selected as the predictor variable for building the simple linear regression model.

Model Building

Simple Linear Regression

A simple linear regression model is used to estimate the relationship between a single predictor variable and a response variable.

Linear Regression Formula

$$y=c+m1x1+m2x2+\cdots+mnxn$$

Where:

- **y** represents the dependent (response) variable.
- **c** is the intercept (constant term).
- **m1** is the coefficient of the first predictor variable.
- **mn** is the coefficient of the nth predictor variable.

Model Used in This Project

Since this analysis uses only **TV advertising** as the predictor variable, the regression equation becomes:

$$Sales=c+m1\times TV$$

Here:

- **Sales** is the target variable.
- **TV** is the independent (predictor) variable.
- **c** is the intercept.
- **m** is the regression coefficient, indicating the change in Sales for every one-unit increase in TV advertising expenditure.

The coefficient values are referred to as **model parameters**, as they define the relationship between the predictor and the response variable.

Generic Steps in model building using statsmodels.

We first assign the feature variable, `TV`, in this case, to the variable `X` and the response variable, `Sales`, to the variable `y`.

```
[26]: X = advertising['TV']
      y = advertising['Sales']
```

The dataset is split into training and testing sets using `train_test_split` from the `sklearn.model_selection` library. In this project, **70%** of the data is used for training the model, while the remaining **30%** is used for testing and evaluation.

```
[26]: X = advertising['TV']
      y = advertising['Sales']
```

```
[29]: from sklearn.model_selection import train_test_split
      X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.7, test_size = 0.3, random_state = 100)
```

```
[30]: # Let's now take a look at the train dataset

      X_train.head()
```

```
[27]: 74     213.4
      3      151.5
      185    205.0
      26     142.9
      90     134.3
      Name: TV, dtype: float64
```

```
[31]: y_train.head()
```

```
[28]: 74     17.0
      3      16.5
      185    22.6
      26     15.0
      90     14.0
      Name: Sales, dtype: float64
```

Building the Linear Regression Model

To build the linear regression model, the `statsmodels.api` library is imported. This library provides the necessary functions to fit the model and generate detailed statistical summaries for analysis.

[34]:
```
import statsmodels.api as sm
```

A constant term is added to the predictor variable using `add_constant()` to estimate the intercept. The linear regression model is then trained using the `OLS` (Ordinary Least Squares) method provided by the `statsmodels` library.

[35]:
```
# Add a constant to get an intercept
X_train_sm = sm.add_constant(X_train)

# Fit the resgression line using 'OLS'
lr = sm.OLS(y_train, X_train_sm).fit()
```

[36]:
```
# Print the parameters, i.e. the intercept and the slope of the regression line fitted
lr.params
```

[32]:
```
const    6.948683
TV       0.054546
dtype: float64
```

[37]:
```
# Performing a summary operation lists out all the different parameters of the regression line fitted
print(lr.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Sales   R-squared:                       0.816
Model:                            OLS   Adj. R-squared:                  0.814
Method:                 Least Squares   F-statistic:                     611.2
Date:                Tue, 14 Jul 2026   Prob (F-statistic):           1.52e-52
Time:                        20:56:43   Log-Likelihood:                -321.12
No. Observations:                 140   AIC:                             646.2
Df Residuals:                     138   BIC:                             652.1
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          6.9487      0.385     18.068      0.000       6.188       7.709
TV             0.0545      0.002     24.722      0.000       0.050       0.059
==============================================================================
Omnibus:                        0.027   Durbin-Watson:                   2.196
Prob(Omnibus):                  0.987   Jarque-Bera (JB):                0.150
Skew:                          -0.006   Prob(JB):                        0.928
Kurtosis:                       2.840   Cond. No.                         328.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

Model Interpretation

The key metrics used to evaluate the regression model include:

- Regression coefficient and its **p-value**
- **R-squared** value
- **F-statistic** and its corresponding **p-value**

1. Regression Coefficient

The coefficient for **TV advertising** is **0.054**, with an extremely small **p-value**. This indicates that TV advertising has a statistically significant positive effect on Sales, suggesting that the observed relationship is unlikely to have occurred by chance.

2. R-squared Value

The model achieves an **R-squared value of 0.816**, indicating that **81.6%** of the variation in Sales can be explained by TV advertising expenditure. This reflects a strong fit between the predictor and the target variable.

3. F-statistic

The **F-statistic** is associated with a very small **p-value**, demonstrating that the overall regression model is statistically significant. This confirms that the relationship between TV advertising and Sales is meaningful rather than the result of random variation.

Regression Equation

The regression model is statistically significant, indicating that it provides a good fit to the data. To better understand the relationship between **TV advertising** and **Sales**, the fitted regression line can be visualized.

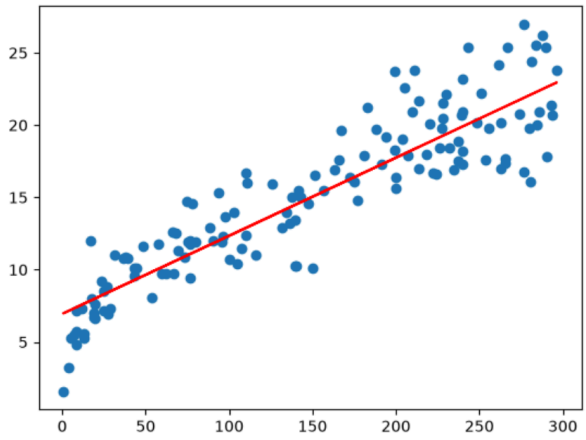
Based on the estimated model parameters, the regression equation is:

Sales = 6.948 + 0.054 × TV

This equation indicates that:

- **6.948** is the intercept, representing the predicted Sales when TV advertising expenditure is zero.
- **0.054** is the regression coefficient, meaning that for every one-unit increase in TV advertising expenditure, the predicted Sales increase by approximately **0.054 units**.

```
[38]: plt.scatter(X_train, y_train)
      plt.plot(X_train, 6.948 + 0.054*X_train, 'r')
      plt.show()
```



Model Evaluation

Residual Analysis

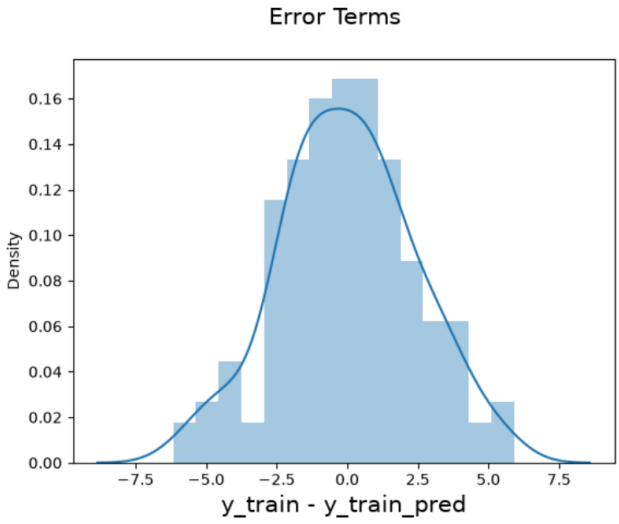
Residual analysis is performed to verify the assumptions of the linear regression model and assess its reliability. Examining the residuals helps determine whether the model provides an appropriate fit for the data.

Distribution of Residuals

One of the key assumptions of linear regression is that the residuals (error terms) are approximately **normally distributed**. To evaluate this assumption, a histogram of the residuals is plotted. If the histogram resembles a normal (bell-shaped) distribution, it indicates that the normality assumption is reasonably satisfied.

[39]:
```
y_train_pred = lr.predict(X_train_sm)
res = (y_train - y_train_pred)
```

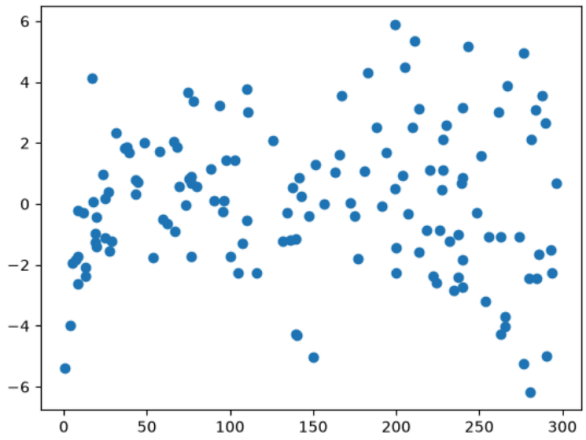
[40]:
```
fig = plt.figure()
sns.distplot(res, bins = 15)
fig.suptitle('Error Terms', fontsize = 15)                  # Plot heading
plt.xlabel('y_train - y_train_pred', fontsize = 15)         # X-label
plt.show()
```



The residuals appear to follow an approximately normal distribution with a mean close to zero, indicating that the normality assumption is satisfied.

Residual Pattern Analysis

```
[41]: plt.scatter(X_train,res)
      plt.show()
```



The regression model is statistically significant and demonstrates satisfactory predictive performance. The approximately normal distribution of the residuals supports the validity of the model assumptions, allowing reliable interpretation of the regression coefficients.

However, the residual plot indicates a slight increase in the spread of residuals as the predictor variable increases, suggesting that some variability in Sales is not fully captured by the model.

Overall, the fitted regression line provides a good representation of the relationship between TV advertising expenditure and Sales, indicating that the model fits the data reasonably well.

Predictions on the Test Set

Once the linear regression model has been trained, predictions are generated for the test dataset. A constant term is first added to `X_test`, after which the trained model uses the `predict()` method to estimate the corresponding Sales values.

```
[42]: # Add a constant to X_test
      X_test_sm = sm.add_constant(X_test)

      # Predict the y values corresponding to X_test_sm
      y_pred = lr.predict(X_test_sm)
```

```
[43]: y_pred.head()
```

```
[39]: 126     7.374140
      104    19.941482
      99     14.323269
      92     18.823294
      111    20.132392
      dtype: float64
```

[44]:
```
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
```

Looking at the RMSE

[50]:
```
#Returns the mean squared error; we'll take a square root
np.sqrt(mean_squared_error(y_test, y_pred))
```

[46]:
```
np.float64(2.0192960089662324)
```

Checking the R-squared on the test set

[46]:
```
r_squared = r2_score(y_test, y_pred)
r_squared
```

[42]:
```
0.7921031601245659
```

Visualizing the fit on the test set

[47]:
```
plt.scatter(X_test, y_test)
plt.plot(X_test, 6.948 + 0.054 * X_test, 'r')
plt.show()
```